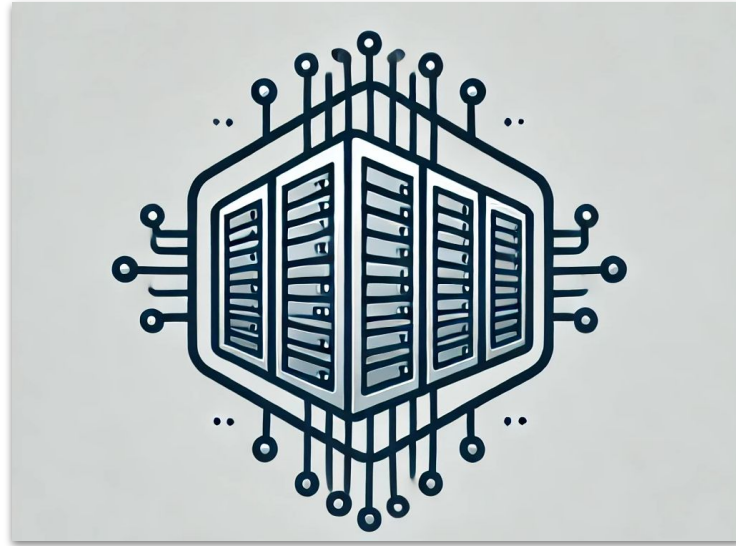


CSC 746: High Performance Computing

CP2 Discussion, Parallelism and Perf Metrics

22 Sep 2026



Today

CP#2

- Presentations: Cordano, Bellenberg
- Discussion

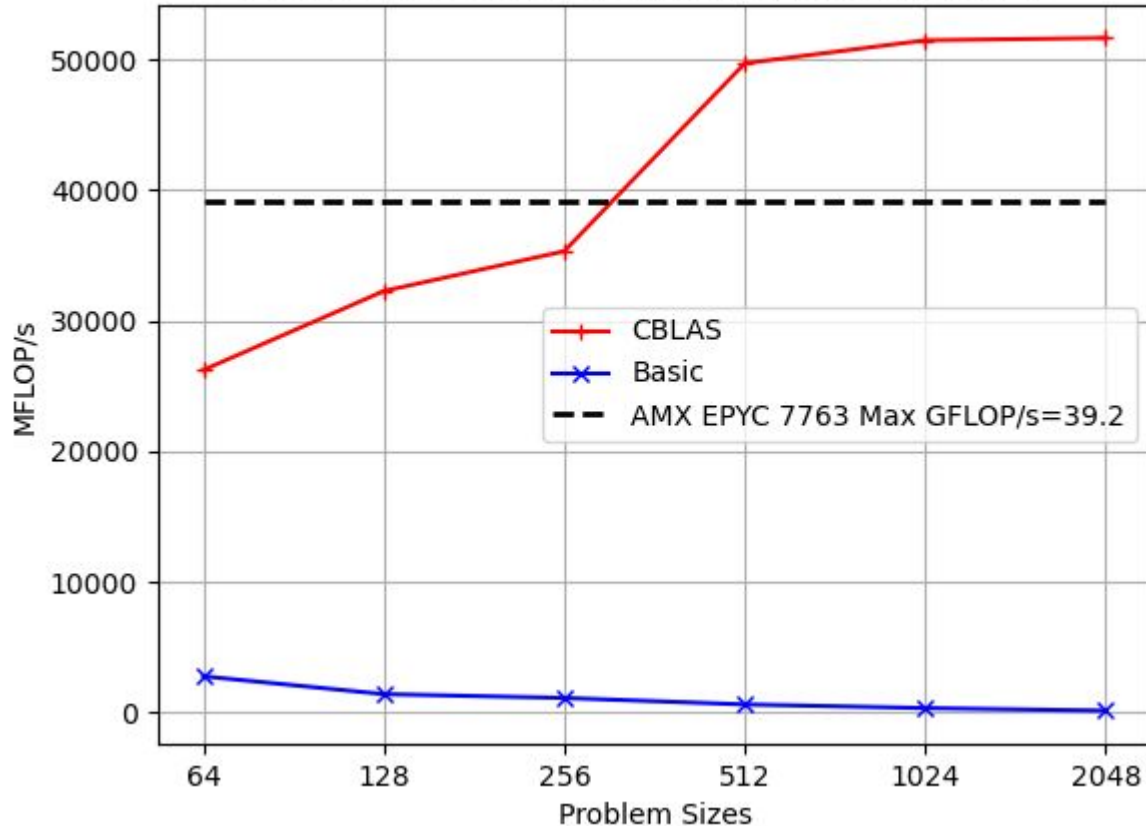
Parallelism and Concurrency

Parallel Performance Measures

Compute time at NERSC is
limited

CP#2 Discussion

MMUL CBLAS vs Basic MFLOP/s, AMD Milan 7763



Main observations:

CBLAS

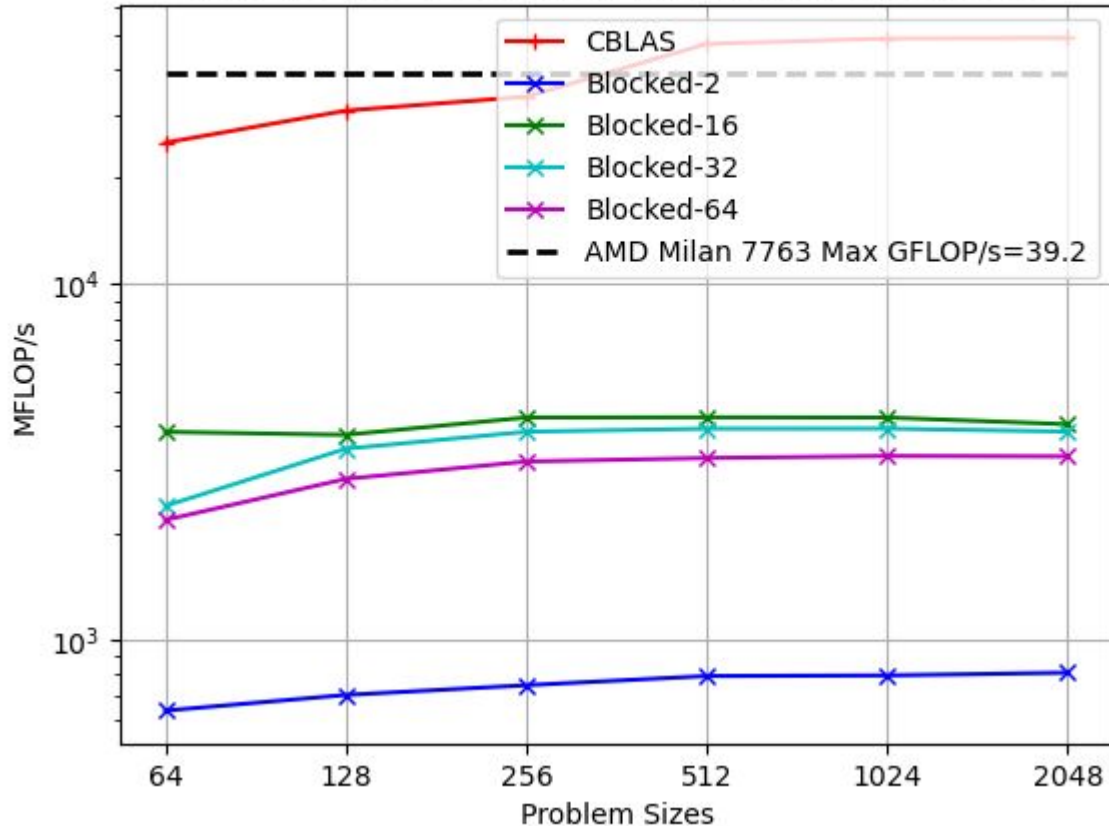
- MFLOP/s increases with problem size up to 512, then flattens out
- At $N \geq 512$, exceeding theoretical peak 39.2 GFLOPS!
- Requires a “Warm-up” run

Basic:

- 2.7 GFLOP/s at $N=64$, ~6% of peak
- 128.0 MFLOP/s min at $N=2048$, 0.3% of peak



MMUL CBLAS vs Blocked MFLOP/s, AMD Milan 7763



Main observations:

Blocked: (except for B=2) relatively steady performance regardless of N

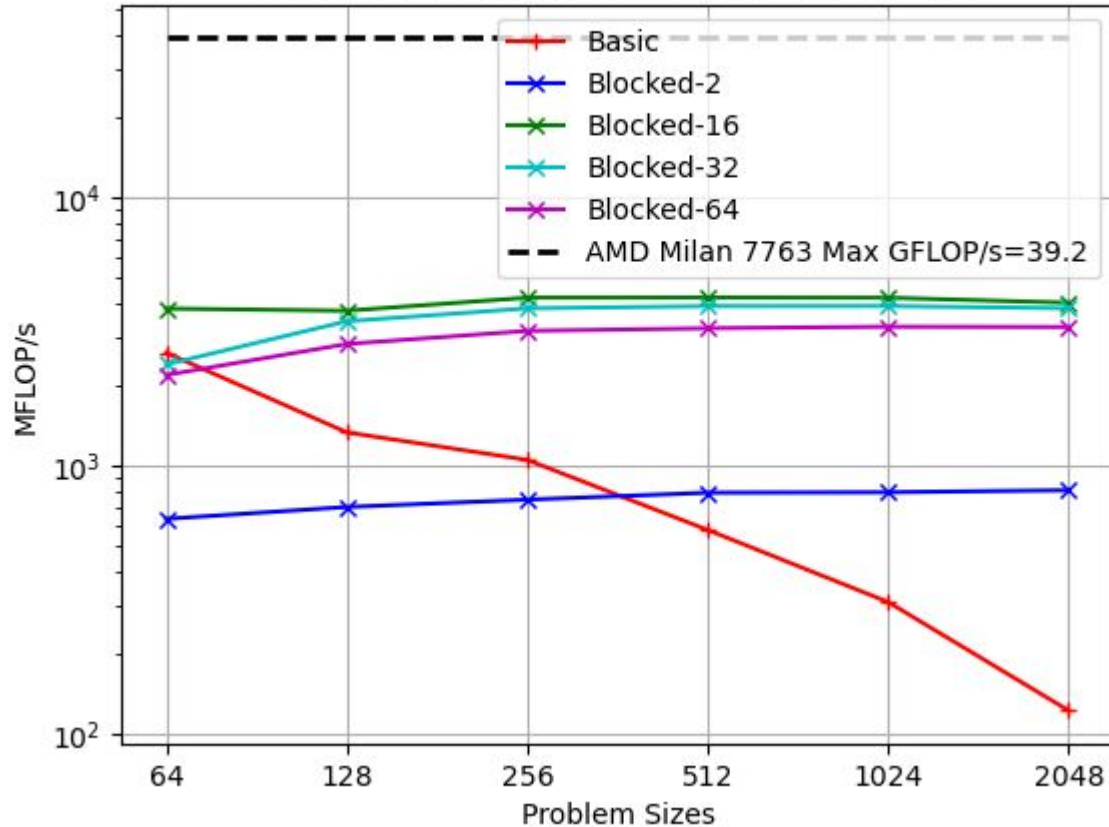
- B=16 → 4.0 GF, 10% of peak
- B=32 → 3.9 GF, 10% of peak
- B=64 → 3.3 GF, 8% of peak

CBLAS: at N=2048, 51.6 GF, 130% of peak !!

Blocking helps to avoid the severe penalties of memory access patterns with large N

Blocking best (4 GFLOP/s, ~10% of peak) nowhere near CBLAS best (51.6 GFLOP/s 130% of peak)

Basic vs Blocked MFLOP/s, AMD Milan 7763



Main observations:

At small problem sizes, basic is competitive (everything fits into cache)

At larger problem sizes, basic quickly becomes non-competitive

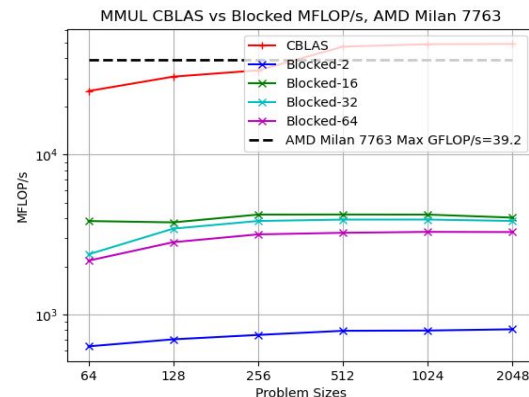
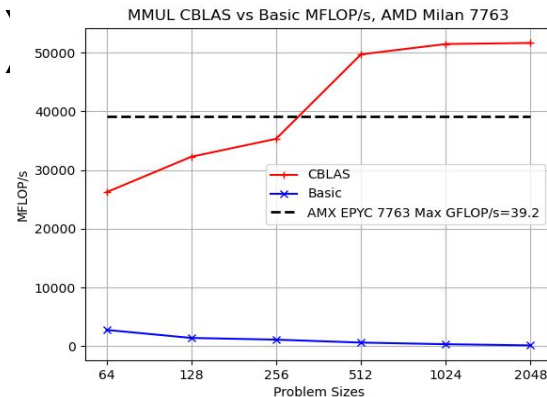
For B=2, lots of overhead copying data around

CP2 Problem Sizes, memory footprint, Perlmutter CPU Node Cache Sizes

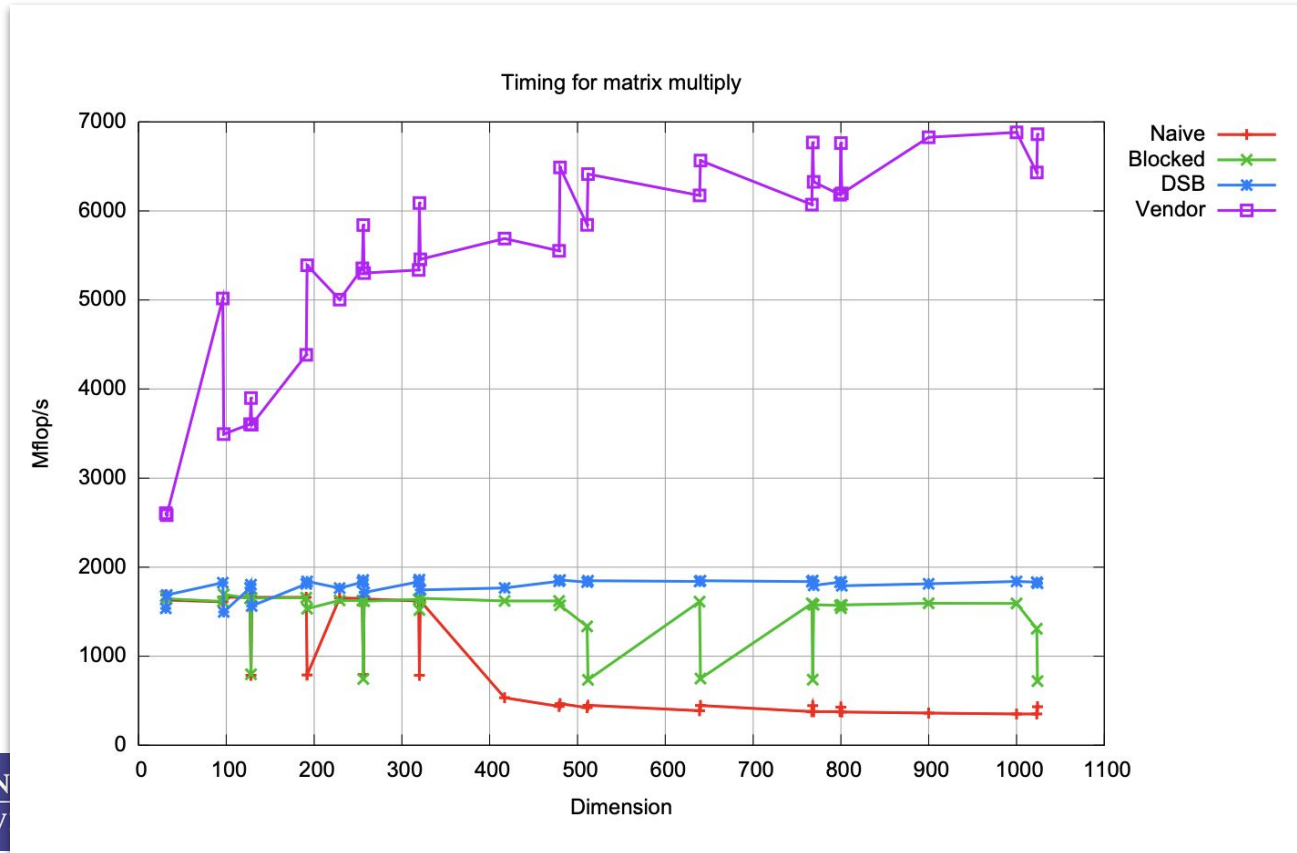
Problem Size	Mem footprint (KiB)	A, B, C mem footprint	L1 cache (KiB)	L2 cache (KiB)	L3 cache (KiB) 32MB * 1024
64	32	96	32	512	32768
128	128	384	32	512	32768
256	512	1536	32	512	32768
512	2048	6144	32	512	32768
1024	8192	24576	32	512	32768
2048	32768	98304	32	512	32768
Block Size	Mem footprint (KiB)	A, B, C mem footprint	L1 cache (KiB)	L2 cache (KiB)	L3 cache (KiB) 256 MB * 1024
2	0.03125	0.09375	32	512	262144
16	2	6	32	512	262144
23	4.1328125	12.3984375	32	512	262144
64	32	96	32	512	262144

Three Questions about CBLAS dgemm()

1. Why is it so much faster than our basic and OpenMP implementations?
2. What MMUL algorithm does BLAS use for its implementation?
3. Is it really exceeding the 39.2 GFLOP/s theoretical peak per-core performance?



CBLAS dgemm() – highly optimized implementation



CBLAS dgemm() – highly optimized implementation

Hand-tuned, processor-specific assembly language code

- AVX2 instructions on AMD EPYC 7763
 - Intel Knights Landing chip had dual AVX-512 units
 - Impossible to access both through g++ and user written C++ code
- Special Fused multiply-add instruction (FMA), 2 FLOPS/clock

Autotuning the block size to match specific processor/cache architecture

- We studied square block sizes
- Would rectangular block shapes perform better?
- What sizes of rectangular blocks might perform best?

Q2. What MMUL algorithm does BLAS use?

Basic MM:

- FLOPS: $2n^3$
- Mem accesses: $n^3 + 3n^2$ (no cache)
 - → Some papers say $4n^2$, but that's not how your implementation works

Blocked:

- FLOPS: $2n^3$
- Mem accesses: $(2N_b + 2) * n^2$, (at $N=1024$, $B=64$, $N_b=16$), $= 18n^2$

Q2. What MMUL algorithm does BLAS use, ctd.?

Strassen's: (1969)

- FLOPS: $O(N^{2.8074})$
- Mem accesses: similar,
 - Access pattern not cache friendly (recursion)
- Replaces some multiplications with additions

- Not as numerically stable as basic/blocked (not used in CBLAS)
- Performs better for Large N , basic does better at small N

1. Basic Idea of Strassen's Algorithm:

- Strassen's algorithm divides two matrices A and B of size $n \times n$ into four submatrices of size $n/2 \times n/2$.
- It computes 7 matrix multiplications (instead of 8 in the classical approach) using a recursive divide-and-conquer method.
- The idea is to trade some scalar multiplications for additional additions and subtractions, which are computationally cheaper than multiplications.

The key steps of Strassen's algorithm reduce the number of multiplications from 8 to 7 for 2×2 blocks and rely on recursion to perform larger multiplications.

Thanks GPT

Q2. What MMUL algorithm does BLAS use, ctd.?

Williams, Xu, Xu, and Zhuo (WXYZ), 2024

- FLOPS $O(N^{2.3728596})$
 - Mem accesses: similar to FLOPS
 - Memory access pattern is not cache friendly: recursion
 - Similar to Strassen's: uses recursion to divide-conquer
 - Additional optimizations to reduce number of FLOPS
-
- Not as numerically stable as basic/blocked (not used in CBLAS)
 - Performs better for Large N, basic does better at small N

Q2. What MMUL algorithm does BLAS use, ctd.?

Williams, Xu, Xu, and Zhuo (WXXZ)

- FLOPS $O(N^{2.3728596})$
- Mem accesses: similar to F
 - Memory access pattern is not
- Similar to Strassen's: uses
- Additional optimizations to
- Not as numerically stable as
- Performs better for Large N

CBLAS most likely uses a variation of your block+copy MMUL

- Hand-optimized assembler
- Leverages vector instructions (eg, AVX2, AVX512)
- Fine-tuning the block size

Name of the game:

- Perform minimum memory accesses while doing the $2N^3$ FLOPS
- Some papers mention $4N^2$ memory accesses

Further reading on MMUL and CBLAS/DGEMM

Further reading:

J. Dongarra et al, A Set of Level 3 Basic Linear Algebra Subprograms, ACM TOMS 16(1), 1990.
<https://dl.acm.org/doi/10.1145/77626.79170>

S. Hadjis, BLAS-level CPU Performance in 100 lines of C-code. <https://cs.stanford.edu/people/shadjis/blas.html>

T. Smith et al., Anatomy of High-performance Many-threaded Matrix Multiplication, IPDPS, 2014.
https://www.cs.utexas.edu/users/flame/pubs/blis3_ipdps14.pdf

R. van de Geijn, E. Quintana-Ortí, [The Science of Programming Matrix Computations](#), 2007

F. van Zee and R. van de Geijn, [BLIS: A Framework for Rapid Instantiation of BLAS Functionality](#), ACM TOMS 2015

Computational Complexity of matrix multiplication ([wikipedia](#))

Q3. Is CBLAS dgemm() exceeding the 39.2 GF peak?

The AMD 7764

39.2 GFLOP/s / 2.45 GHz $\rightarrow$ 16 dp FLOPS / clock

16 dp FLOPS/clock? How?

AVX2 instructions, 256-bit vector instructions, or 16 dp FLOPS/clock

- Each 256-bit AVX2 vector register can hold 4, 8-byte doubles
- Use a “fused multiply add” instruction for 2 FLOPs/clock
- There are two such AVX2 vector units per core
- 4 doubles * 2 FLOPS * 2 FMA units = 16 FLOPS/clock

Q3. Is CBLAS dgemm() exceeding the 39.2 GF peak?

The AMD 7764

AVX2 instructions, 256-bit vector instructions, or 16 dp FLOPS/clock

How to use both FMA units?

- Loop unrolling
- Ensure load/compute instructions don't have contention
- Careful hand-crafted x86 assembly language ...

Does G++ -O3 produce this kind of code?

- Not sure, would have to examine assembly language produced by G++

Q3. CBLAS and the AMD 7763 → 39.2 GFLOP/s peak?

CBLAS: assume $2 \cdot N^3$ FLOPS

Assumption of 39.2 GFLOP/s peak is computed as:

$$2.45 \text{ GHz} * 16 \text{ FLOPS/clock}$$

On the spec sheet:

- Base clock = 2.45 GHz
- Max boost clock = 3.5 GHz

At 3.5 GHz, then the peak is 56 GFLOPS

General Specifications	
Platform:	Server
Product Family:	AMD EPYC™
Product Line:	AMD EPYC™ 7003 Series
# of CPU Cores:	64
# of Threads:	128
Max. Boost Clock🔵:	Up to 3.5GHz
Base Clock:	2.45GHz

Q3. CBLAS and the AMD 7763

CBLAS: assume $2 \cdot N^3$ FLOPS

Assumption of 39.2 GFLOP/s peak is computed as:

$$2.45 \text{ GHz} * 16 \text{ FLOPS/clock}$$

On the spec sheet:

- Base clock = 2.45 GHz
- Max boost clock = 3.5 GHz

At 3.5 GHz, then the peak is 56 GF

- If using 2 FMAs, then
- $3.5 \text{ GHz} * 32 \text{ FLOPS/clock} = 112 \text{ GF peak}$

Conditions when clock rate will boost:

- Workload demand
- Thermal and power conditions
- Number of active cores
- Duration and load
- BIOS settings
- Serial computation (not multi-core)

Could clock boost be happening for us?

- If BIOS settings permit
- Then it is likely

Question: we measure 51 GFLOPS

Q3. Is CBLAS dgemm() exceeding the 39.2 GF peak?

Yes, it clearly is exceeding 39.2 GF: our charts show this clearly

Why?

- 3.5 GHz clock in effect during have serial workload: 56 GF peak
- Likely: using both FMA units with carefully crafted x86 AVX2 assembly code
- It is using a variant of your block+copy optimization
 - Block sizes tuned for the architecture
 - Thoughts about block sizes in CP#2?

Parallelism and Concurrency

What is Parallelism? Concurrency?

Concurrency: a condition of a system in which multiple tasks are *logically* active at one time

Parallelism: a condition of a system in which multiple tasks are actually active at one time

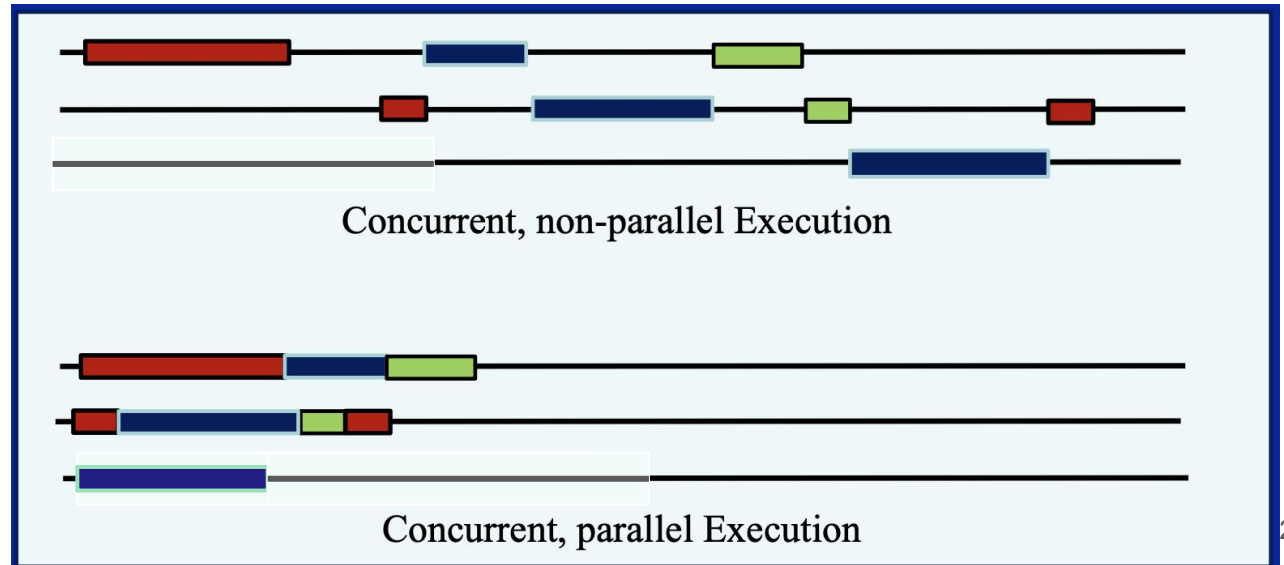


Image Credit: Tim Mattson, Intel
Modified locally for correctness

What is Parallelism? Concurrency?

Concurrency: a condition of a system in which multiple tasks are *logically* active at one time

Parallelism: a condition of a system in which multiple tasks are actually active at one time

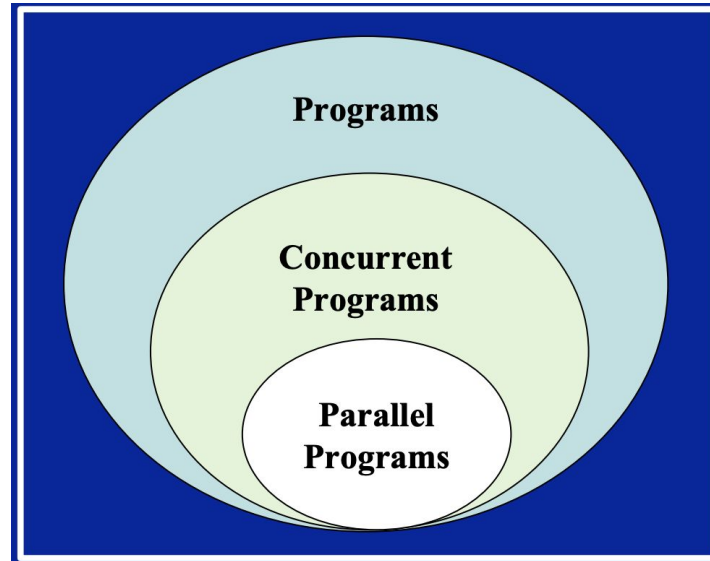


Image Credit: Tim Mattson, Intel

Concurrency in Action: The Web Server

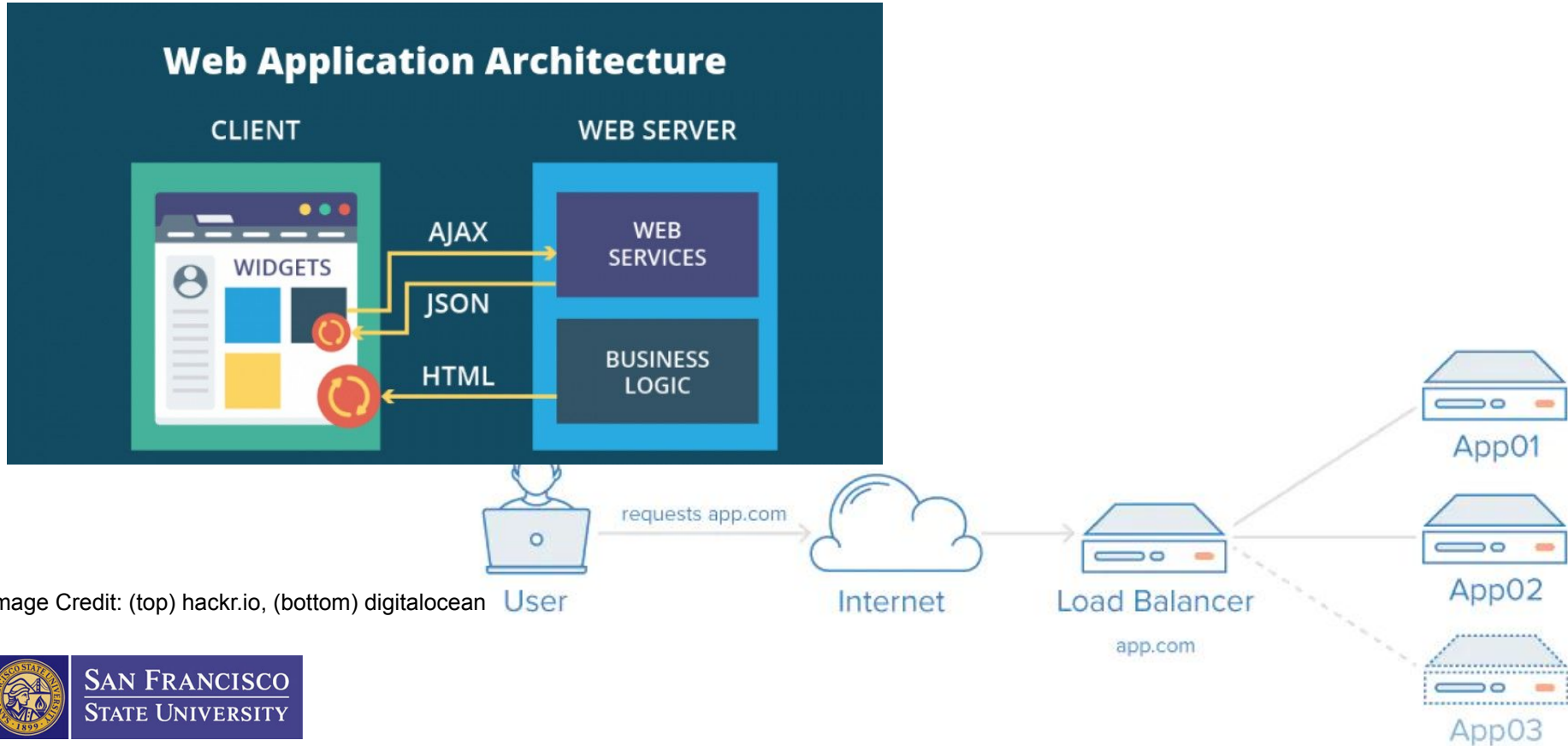


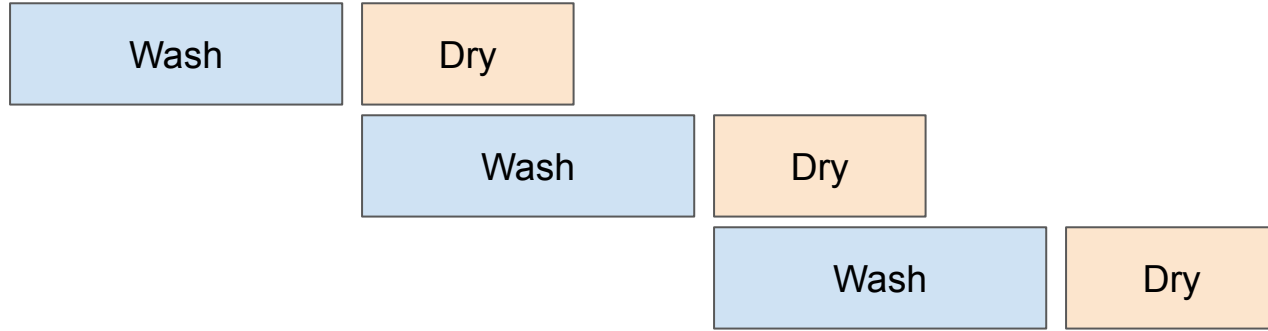
Image Credit: (top) hackr.io, (bottom) digitalocean



SAN FRANCISCO
STATE UNIVERSITY

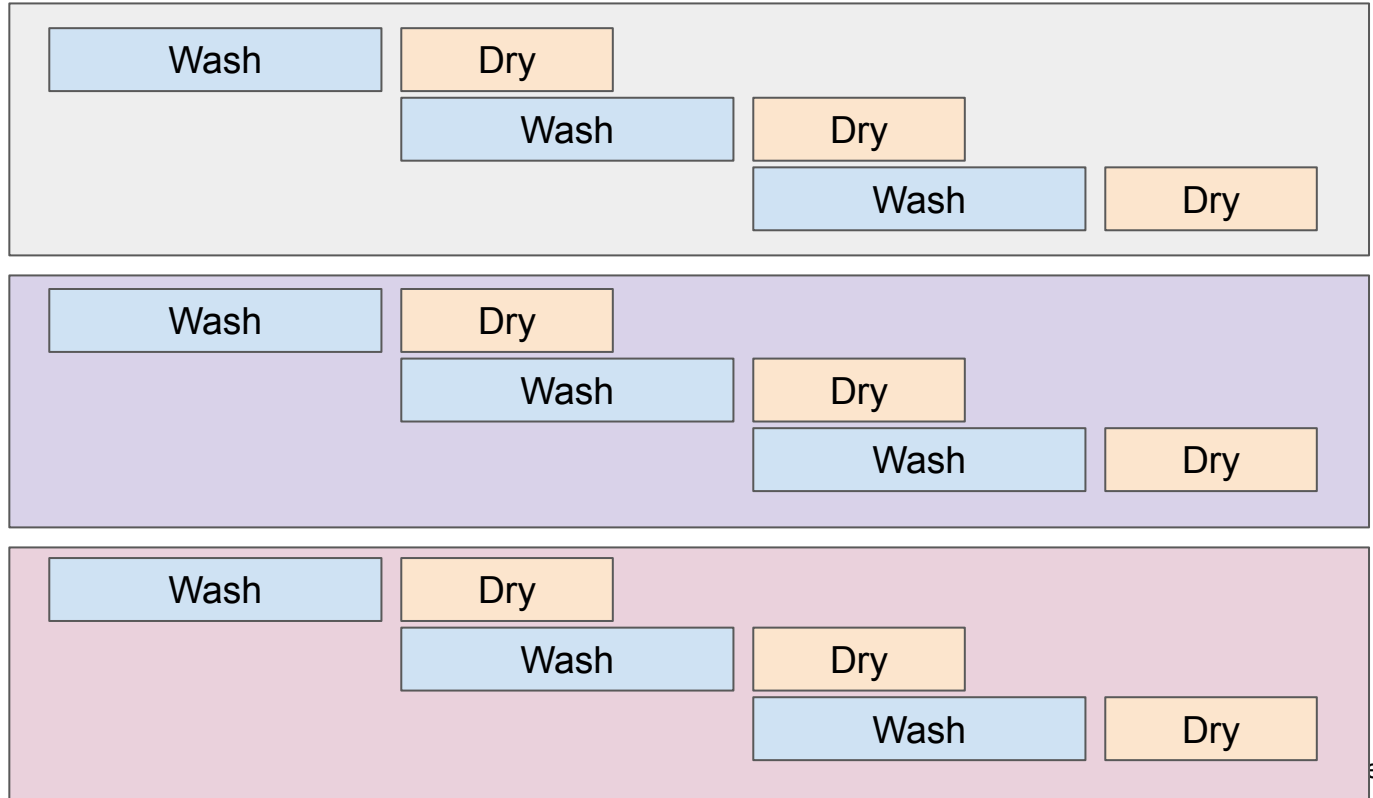
Pipelining vs. Concurrency vs. Parallelism

Laundry:
2-stage
pipeline

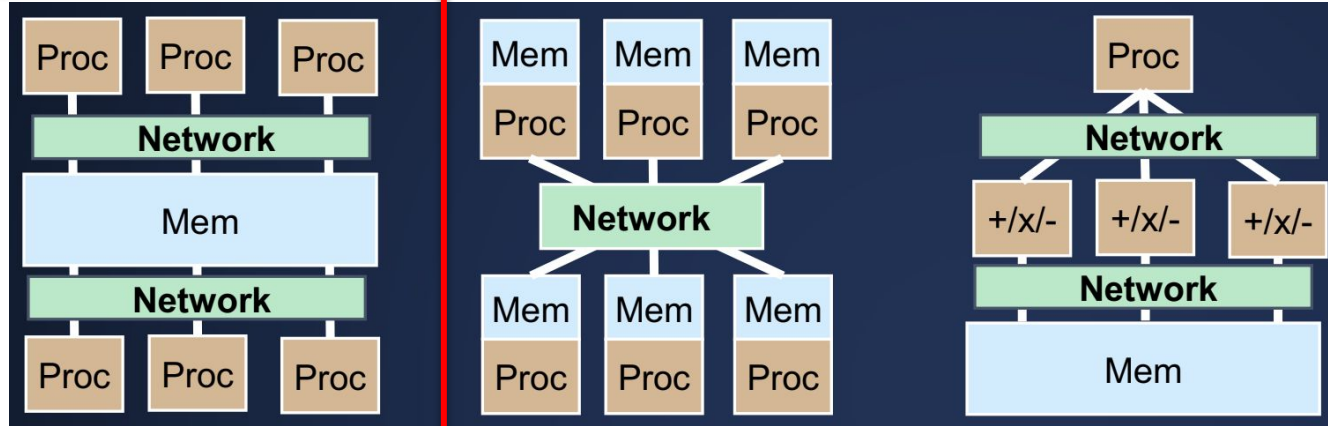


Pipelining vs. Concurrency vs. Parallelism

Laundromat:
N-way
(potential)
parallelism



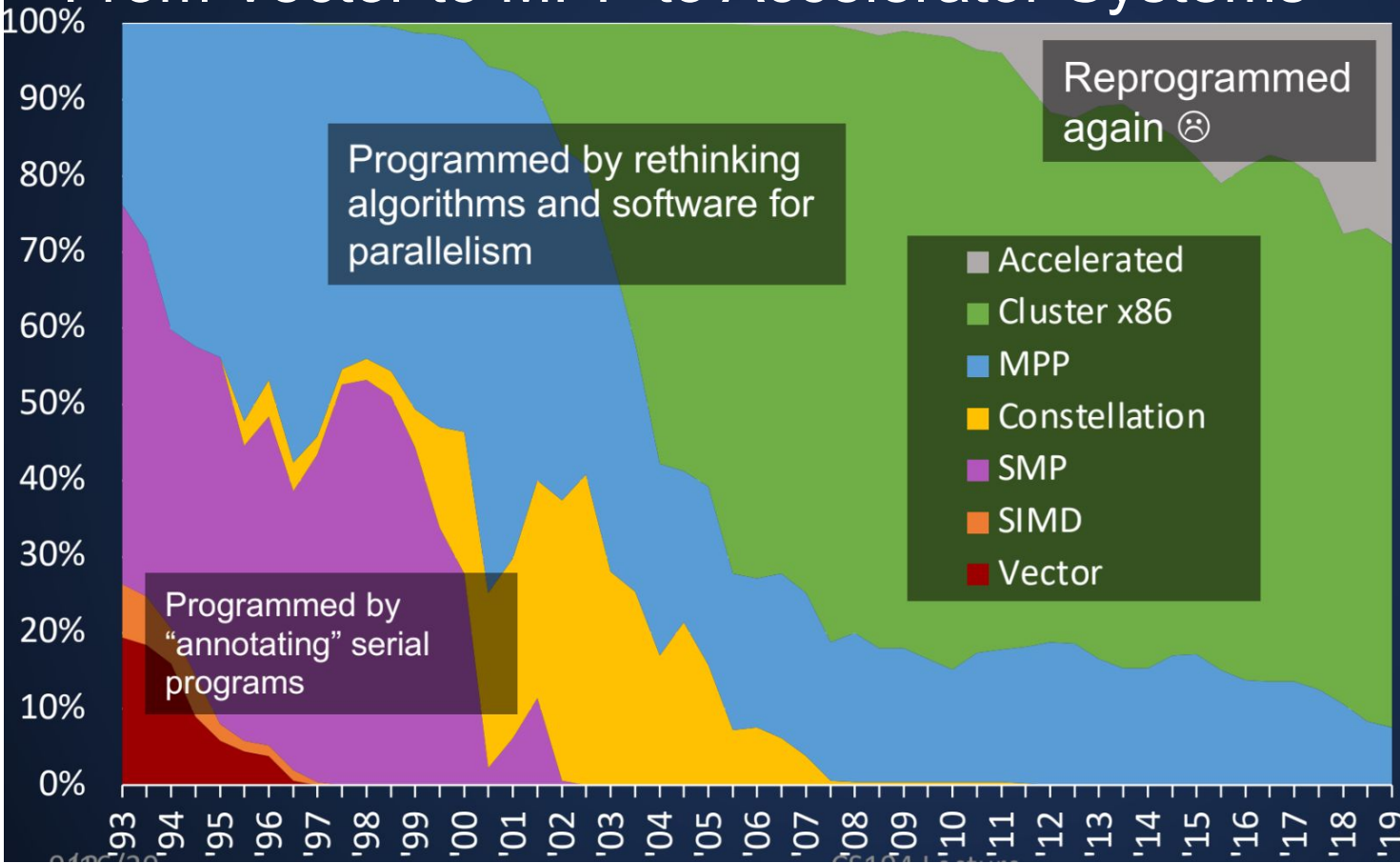
Parallel Machine Models



Abstract
machine
models

Shared memory	Distributed memory	SIMD (Vector)
Processors execute own instruction stream	Processors execute own instruction stream	One instruction stream applied to multiple data
Communicate by reading/writing memory	Communicate by sending messages	Communicate through memory

From Vector to MPP to Accelerator Systems



Brief History of Parallel “Languages”

Vector machine era (Cray 1, XMP, YMP, etc.)

- Parallel “languages” were loop annotations to specify vectorization
- Performance was fragile, good user support

Brief History of Parallel “Languages”

Vector machine era (Cray 1, XMP, YMP, etc.)

- Parallel “languages” were loop annotations to specify vectorization
- Performance was fragile, good user support

SIMD machine era (Connection Machine)

- Data parallel languages popular and successful (CMF, C*, *Lisp, ...)
- Irregular data (sparse mat-vec multiply OK), but irregular computation (adaptive meshes, divide-conquer) less clear



Brief History of Parallel “Languages”

Vector machine era (Cray 1, XMP, YMP, etc.)

- Parallel “languages” were loop annotations
- Performance was fragile, good user support

SIMD machine era (Connection Machine)

- Data parallel languages popular and successful (CMF, C*, *Lisp, ...)
- Irregular data (sparse mat-vec multiply OK), but irregular computation (adaptive meshes, divide-conquer) less clear

SMP era (SGI Onyx, etc.)

- Shared memory programming models: fork, POSIX threads, OpenMP (task parallelism, data parallelism)



Brief History of Parallel “Languages”

Vector machine era (Cray 1, XMP, YMP, etc.)

- Parallel “languages” were loop annotations
- Performance was fragile, good user support

SIMD machine era (Connection Machine)

- Data parallel languages popular and successful (CMF, C*, *Lisp, ...)
- Irregular data (sparse mat-vec multiply OK), but irregular computation (adaptive meshes, divide-conquer) less clear

SMP era (SGI Onyx, Cray S-MP, etc.)

- Shared memory programming models: fork, POSIX threads, OpenMP

Clusters era (Cray T3E, CM5, ...)

- The Message Passing Interface (MPI) became dominant
 - You divide up the problem, coordinate data movement and synchronization
 - Data parallelism (mostly)



Brief History of Parallel “Languages”

Vector machine era (Cray 1, XMP, YMP, etc.)

- Parallel “languages” were loop annotations
- Performance was fragile, good user support

SIMD machine era (Connection Machine)

- Data parallel languages popular and successful (CMF, C*, *Lisp, ...)
- Irregular data (sparse mat-vec multiply OK), but irregular computation (adaptive meshes, divide-conquer) less clear

SMP era (SGI Onyx, Cray S-MP, etc.)

- Shared memory programming models: fork, POSIX threads, OpenMP

Clusters era (Cray T3E, CM5, ...)

- The Message Passing Interface (MPI) became dominant
 - You divide up the problem, coordinate data movement and synchronization
 - Data parallelism (mostly)

Addition of accelerators (GPU, FPGA)

- CUDA, OpenACC, ..
- Data parallelism, disjoint memory spaces

Brief History of Parallel “Languages”

Vector machine era (Cray 1, XMP, YMP, etc.)

- Parallel “languages” were loop annotations
- Performance was fragile, good user support

SIMD machine era (Connection Machine)

- Data parallel languages popular and successful (CMF, C*, *Lisp, ...)
- Irregular data (sparse mat-vec multiply OK), but irregular computation (adaptive meshes, divide-conquer) less clear

SMP era (SGI Onyx, Cray S-MP, etc.)

- Shared memory programming models: fork, POSIX threads, OpenMP

Clusters era (Cray C90, CM5, ...)

- The Message Passing Interface (MPI) became dominant

Addition of accelerators (GPU, FPGA)

- CUDA, OpenACC, ..
- Data parallelism, disjoint memory spaces

Cloud computing

- Hadoop, SPARK (task parallelism)

The End



A Warner Bros.
PICTURE

